

Aufgabe 1: Ankreuzfragen (24 Punkte)

1) Einfachauswahlfragen (16 Punkte)

Bei den Einfachauswahlfragen in dieser Aufgabe ist jeweils nur **eine** richtige Antwort eindeutig anzukreuzen. Auf die richtige Antwort gibt es die angegebene Punktzahl.

Wollen Sie eine Antwort korrigieren, streichen Sie bitte die falsche Antwort mit drei waagrechten Strichen durch (~~☒~~) und kreuzen die richtige an.

Lesen Sie die Frage genau, bevor Sie antworten.

a) Gegeben seien die folgenden Präprozessor-Makros:

```
#define SUB(a, b) a - b
```

```
#define DIV(a, b) a / b
```

Was ist das Ergebnis des folgenden Ausdrucks? $3 * DIV(SUB(12, 6), 3)$

2 Punkte

- ☐ 34
☐ 30
☐ 10
☐ 6

b) Welche Aussage zu Zeigern ist richtig?

2 Punkte

- ☐ Zeiger können in C nicht als Parameter an Funktionen übergeben werden.
☐ Der Compiler erkennt bei der Verwendung eines ungültigen Zeigers die problematische Code-Stelle und generiert Code, der zur Laufzeit die Meldung „Segmentation fault“ ausgibt.
☐ Zeiger können verwendet werden, um in C eine call-by-reference Übergabesemantik nachzubilden.
☐ Ein Zeiger kann zur Manipulation von schreibgeschützten Datenbereichen verwendet werden.

c) Welche der folgenden Aussagen zum Thema Threads und Prozesse ist richtig?

2 Punkte

- ☐ Threads (*light-weight processes*) blockieren sich bei blockierenden Systemaufrufen gegenseitig.
☐ Zu jedem Thread (*light-weight process*) gehört ein eigener, isolierter Adressraum.
☐ Die Umschaltung von User-Level Threads (*feather-weight processes*) ist eine privilegierte Operation und muss deshalb im Systemkern erfolgen.
☐ Threads (*light-weight processes*) teilen sich den Adressraum mit Ausnahme des Stack-Segments.

d) Ein Prozess wird vom Zustand blockiert (*blocked*) in den Zustand bereit (*ready*) überführt. Welche Aussage passt zu diesem Vorgang?

2 Punkte

- ☐ Der Prozess hat auf Daten von der Festplatte gewartet und die Daten stehen nun zur Verfügung.
☐ Der Prozess wird wegen eines ungültigen Speicherzugriffs (*segmentation fault*) beendet.
☐ Es ist kein direkter Übergang von dem Zustand blockiert (*blocked*) in den Zustand bereit (*ready*) möglich.
☐ Der Prozess wird erstmalig gestartet.

e) Welche Aussage zum Thema *Inter-Process Communication (IPC)* ist richtig?

2 Punkte

- ☐ Beim nachrichten-basierten (*message-based*) IPC müssen sowohl Sender als auch Empfänger immer blockieren bis die Nachricht von der Gegenseite bestätigt wurde.
☐ Beim IPC basierend auf geteiltem Speicher (*shared memory*) wird spezielle Hardware verwendet, sodass keine Synchronisation der Speicherzugriffe nötig ist.
☐ Beim nachrichten-basierten (*message-based*) IPC innerhalb eines Computernetzwerks können Sockets als Kommunikationsendpunkte (*communication endpoints*) genutzt werden.
☐ Bei IPC zwischen Prozessen verschiedener Systeme werden Unix Pipes und Signale zur Kommunikation verwendet.

f) Welche Aussage zu Adressräumen von UNIX-Prozessen ist richtig?

2 Punkte

- ☐ Konstante Zeichenketten liegen im Daten-Segment.
☐ Auf Variablen im Daten-Segment kann nur lesend zugegriffen werden.
☐ Variablen der Speicherklasse *static* liegen im Daten-Segment.
☐ Alle Variablen im Stack-Segment werden durch den Übersetzer (*Compiler*) mit dem Wert 0 initialisiert.

g) Welche Aussage zum Translation Lookaside Buffer (TLB) ist richtig?

2 Punkte

- ☐ Bei einem Kontextwechsel (*context switch*) müssen in der Regel keine Einträge des TLB ersetzt werden.
☐ Der TLB puffert die Ergebnisse der Abbildung von physikalischen auf logische Adressen, sodass eine erneute Anfrage sofort beantwortet werden kann.
☐ Wird eine Speicherabbildung im TLB nicht gefunden, wird der auf den Speicher zugreifende Prozess mit einer Schutzraumverletzung (*Segmentation Fault*) abgebrochen.
☐ Wird eine Speicherabbildung im TLB nicht gefunden, wird die Abbildung in den Seitentabellen (*page tables*) nachgeschlagen und im TLB eingetragen.

h) Welche Aussage zu prioritätsbasierten Scheduling-Verfahren ist richtig?

2 Punkte

- ☐ Prioritätsumkehr (*priority inversion*) meint, dass ein Prozess mit höherer Priorität durch einen Prozess mit niedrigerer Priorität blockiert wird, wenn dieser eine Ressource exklusiv belegt.
- ☐ Prioritätsumkehr (*priority inversion*) kann nur mit dynamischen Prioritäten auftreten.
- ☐ Beim prioritätsbasierten Scheduling ist ein Verhungern (*starvation*) von Prozessen ausgeschlossen.
- ☐ Der Prozess mit der höchsten Priorität wird niemals vom Scheduler unterbrochen.

2) Mehrfachauswahlfragen (8 Punkte)

Bei den Mehrfachauswahlfragen in dieser Aufgabe sind jeweils m Aussagen angegeben, davon sind n ($0 \leq n \leq m$) Aussagen richtig. Kreuzen Sie alle richtigen Aussagen an. Jede korrekte Antwort in einer Teilaufgabe gibt einen Punkt, jede falsche Antwort einen Minuspunkt. Eine Teilaufgabe wird minimal mit 0 Punkten gewertet, d. h. falsche Antworten wirken sich nicht auf andere Teilaufgaben aus.

Wollen Sie eine falsch angekreuzte Antwort korrigieren, streichen Sie bitte das Kreuz mit drei waagrechten Strichen durch (~~☒~~).

Lesen Sie die Frage genau, bevor Sie antworten.

a) Welche Aussagen zum Thema Nebenläufigkeit und Synchronisation sind richtig?

4 Punkte

- ☐ Durch den Einsatz von Semaphoren kann ein wechselseitiger Ausschluss erzielt werden.
- ☐ Die V-Operation kann auf einem Semaphor nur von dem Thread aufgerufen werden, der zuvor auch die P-Operation aufgerufen hat.
- ☐ Die Nutzung von Semaphoren zum Erreichen eines wechselseitigen Ausschlusses verhindert grundsätzlich Verklemmungen (*deadlocks*).
- ☐ Einseitige Synchronisation (*unilateral synchronisation*) kann zur Signalisierung der Verfügbarkeit von Ressourcen genutzt werden.
- ☐ Die P-Operation eines Semaphors erhöht den Wert des Semaphors um 1 und deblockiert gegebenenfalls wartende Prozesse.
- ☐ Ein Semaphor kann nur zur Signalisierung von Ereignissen, nicht jedoch zum Erreichen eines wechselseitigen Ausschlusses verwendet werden.
- ☐ Mehrseitige Synchronisation (*multilateral synchronisation*) kann zur Zugriffs-koordination geteilter Ressourcen genutzt werden.
- ☐ Eine Verklemmung (*deadlock*) kann nur dann auftreten, wenn Prozesse auch zirkulär aufeinander warten (*circular waiting condition*).

b) Welche der folgenden Aussagen zu UNIX-Dateisystemen sind richtig?

4 Punkte

- ☐ Im Wurzelverzeichnis '/' existiert kein Eintrag '..'.
- ☐ Ein Pfadname, der nicht mit einem '/'-Zeichen beginnt, wird relativ zum Home-Verzeichnis des Benutzers interpretiert.
- ☐ In den Attributen einer Inode wird ein Referenzzähler mit der Anzahl der *symbolic links*, die auf die Inode verweisen, gespeichert.
- ☐ Im Wurzelverzeichnis '/' verweist der Eintrag '..' wieder auf das Wurzelverzeichnis.
- ☐ In jedem Verzeichnis gibt es einen Eintrag, der auf das Verzeichnis selbst verweist.
- ☐ Beim Anlegen einer Datei wird die maximale Größe festgelegt. Wird sie bei einer Schreiboperation überschritten, wird ein Fehler gemeldet.
- ☐ In den Attributen einer Inode werden Dateityp, Eigentümer und Dateigröße gespeichert.
- ☐ Innerhalb eines Verzeichnisses können mehrere Verweise auf dieselbe Inode existieren, sofern diese unterschiedliche Namen haben.

Aufgabe 2: dips (45 Punkte)

Sie dürfen diese Seite zur besseren Übersicht bei der Programmierung heraustrennen!

Schreiben Sie ein POSIX-1.2008 und C11-konformes Programm **dips** (**D**ata **I**ntegrity **P**rotection Software), welches die Integrität von Dateien basierend auf SHA1-Hashes überprüft. Das Programm liest zunächst aus einer Auftragsdatei eine Liste von zu überprüfenden Dateien und den jeweils erwarteten Hashes ein. **dips** berechnet für jede Datei den SHA1-Hash und vergleicht, ob dieser mit dem in der Auftragsdatei angegebenen Hash übereinstimmt. Da die Berechnung der Hashes rechenintensiv sein kann, soll diese Berechnung durch die Nutzung von Threads parallelisiert werden. Am Programmende wird ausgegeben, bei wie vielen Dateien die Hashes nicht übereingestimmt haben.

Die Dateinamen und erwarteten Hashwerte sind in der Auftragsdatei `checksums.txt` gespeichert. Die Auftragsdatei enthält einen Eintrag pro Zeile. Am Anfang der Zeile steht der Name der Datei, gefolgt von dem SHA1-Hash, der für diese Datei erwartet wird. Der Dateiname und der Hash sind durch ein einzelnes Slash-Zeichen ('/') getrennt. Sie dürfen davon ausgehen, dass jeder Dateiname maximal 256 Zeichen lang ist und kein Slash-Zeichen enthält. Die Hex-Repräsentation eines SHA1-Hashes ist immer exakt 40 Zeichen lang. Zudem ist sichergestellt, dass die Auftragsdatei nicht mehr als 200 Zeilen lang ist, dass jede Zeile korrekt formatiert ist und jede Zeile Daten enthält.

Beispielhafter Inhalt der Auftragsdatei `checksums.txt`:

```
sem.c/e9a77bdee583f811a4fb525f77b2a6335bad28bc
sem.h/df44b086a1719419adb46e74648fd8eefee05859
```

Das Programm **dips** soll folgendermaßen strukturiert sein:

- Funktion **int main(void)**:
Ruft `parse_file` (siehe unten) zum Einlesen von `checksums.txt` auf und initialisiert benötigte Datenstrukturen. Anschließend werden die Hashes der Dateien berechnet und mit dem jeweils erwarteten Hash verglichen. Dazu wird pro Datei ein Thread gestartet, der die Funktion `thread_start` (siehe unten) ausführt. Es sollen maximal 10 Threads parallel laufen und jeweils im `detached` Modus ausgeführt werden.

Der Hauptthread wartet bis sich alle Threads beendet haben und gibt dann die Anzahl der Dateien aus, bei denen der berechnete Hash nicht mit dem erwarteten Hash übereingestimmt hat. Die Anzahl der fehlerhaften Hash-Werte wird von den Threads in einer globalen Variable verwaltet. Abschließend werden die angeforderten Ressourcen (inkl. der aus `parse_file`) freigegeben.
- Funktion **size_t parse_file(void)**:
Liest die Auftragsdatei `checksums.txt` zeilenweise ein. Die Auftragsdatei liegt im Ausführungsordner von **dips**. Die eingelesenen Daten werden in ein globales Array bestehend aus `file_t`-Einträgen gespeichert. Die Funktion gibt die Zahl der eingelesenen Einträge zurück.
- Funktion **void *thread_start(void *sfile)**:
Bekommt einen `file_t`-Zeiger übergeben und berechnet den SHA1-Hash für die übergebene Datei. Nutzen Sie hierfür die Funktion **bool sha1(char *file, char *hash)** der Schnittstelle `sha1.h`. Tritt bei der Hashberechnung ein Fehler auf, wird das Programm mit einer Fehlermeldung beendet. Danach wird der berechnete Hash mit dem erwarteten Wert verglichen. Wenn die Hashes nicht übereinstimmen, wird die Zahl der fehlerhaften Werte um 1 erhöht.

Hinweise:

- Die letzte Zeile endet nicht mit einem `\n`.
- Achten Sie auf die korrekte Synchronisation geteilter Daten.
- Für die Ausgabe ist keine Fehlerbehandlung nötig.
- Achten Sie ansonsten auf korrekte und vollständige Fehlerbehandlung.

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <errno.h>

#include "sem.h"
#include "sha1.h"

static void die(const char msg[]) {
    perror(msg);
    exit(EXIT_FAILURE);
}

static void err(const char msg[]) {
    fprintf(stderr, "%s", msg);
    exit(EXIT_FAILURE);
}

typedef struct file {
    char* name;
    char* hash;
} file_t;

// Makros, Funktionsdeklarationen, globale Variablen
// ...

// Funktion main
// ...

// Auftragsdatei einlesen und Initialisierung
// ...

// ...
```

```
// ...

// Threads starten
// ...

// Warten auf Threads und Ergebnis ausgeben
// ...

// Aufräumen
// ...

// Ende Funktion main
```

```
// Funktion parse_file
```

10

```
// Zeilenweises Einlesen der Datei
```

7

```
// Eintrag parsen und in Array einfügen
```

11

7

```
// Aufräumen
```

```
// Ende Funktion parse_file
```

P:

// Funktion thread_start

// Hash-Wert berechnen und überprüfen

// Ende Funktion thread_start

T:

Vervollständigen Sie das folgende Makefile. Es soll ein Target dips unterstützt werden, welches das Programm dips aus der Quelldatei dips.c und den geforderten Modulen baut. Greifen Sie dabei stets auf Zwischenprodukte (z.B. dips.o) zurück. Gehen Sie davon aus, dass die vorgegebenen Module sem.o und sha1.o stets vorliegen und daher nicht erzeugt werden müssen.

Nutzen Sie dabei die Variablen CC und CFLAGS konventionskonform. Achten Sie darauf, dass das Makefile ohne eingebaute Variablen und Regeln (Aufruf von make -Rr) funktioniert!

.PHONY: all clean

CC=gcc

CFLAGS=-std=c11 -pedantic -Wall -D_XOPEN_SOURCE=700 -pthread

all: dips

clean:

rm -f dips dips.o

Mk:

Aufgabe 3: Prozesse & Adressräume (9 Punkte)

1) Gegeben sei das nachfolgende Programm. Vervollständigen Sie den Aufbau des logischen Adressraums eines Prozesses, der dieses Programm ausführt. Gehen Sie von einem x86-System aus. (7 Punkte)

- Ergänzen Sie in der unten stehenden Zeichnung die fehlenden Segmente und deren Namen (analog zum schon vorgegebenen Textsegment). Unterscheiden Sie hierbei die Bereiche zur Speicherung von initialisierten und nicht initialisierten Variablen. Ergänzen Sie die Wachstumsrichtung bei Segmenten variabler Größe. (3 Punkte)
- Ordnen Sie alle im Quelltext vorkommenden Variablen und Funktionen dem jeweiligen Segment zu. Markieren Sie für Zeigervariablen mittels Pfeil, auf welches Datum der Zeiger jeweils zeigt (4 Punkte)

```
#define SIZE 16
```

```
int b = 0;
```

```
const char *s = "Hello\n";
```

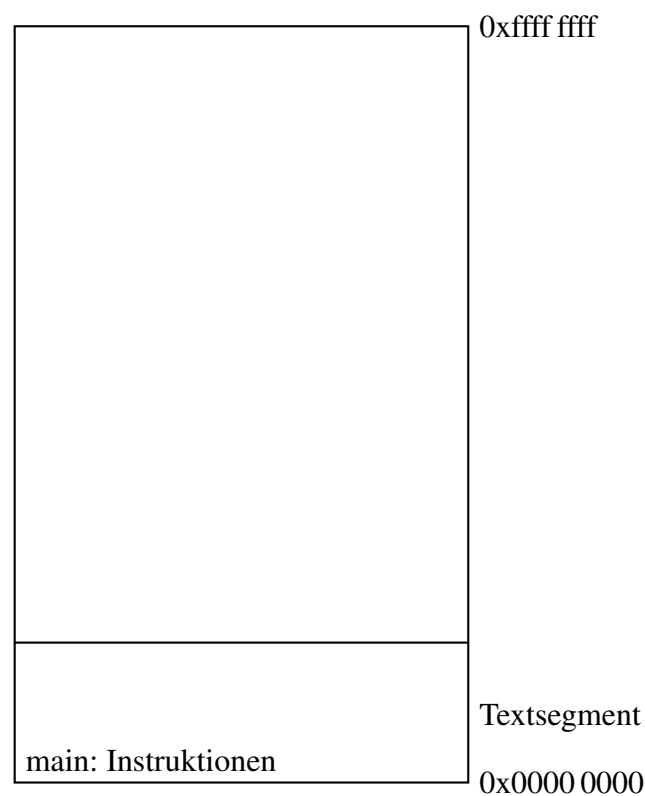
```
int main(void) {
    void *arr = malloc(SIZE);
```

```
int i = 7;
int *j = &i;
```

// ...

}

```
void run(void) {
    // ...
}
```



2) In welcher Struktur wird der Prozess Kontext (*process context*) verwaltet? Nennen Sie die Struktur und drei weitere Kontextelemente, neben dem logischen Adressraum, die in der Struktur verwaltet werden. (2 Punkte)

Klausur Betriebssysteme

Aufgabe 4: Scheduling (12 Punkte)

1) Worin unterscheiden sich kooperatives (*cooperative*) und präemptives (*preemptive*) Scheduling? Nennen Sie zusätzlich je einen Vorteil der beiden Schedulingarten. (4 Punkte)

2) Erklären Sie stichpunktartig die Funktionsweise des Round Robin Algorithmus im präemptiven (*preemptive*) Scheduling. Gehen Sie insbesondere darauf ein, welche Aktionen bei der Verdrängung eines Prozesses durchgeführt werden. (3 Punkte)

3) Erläutern Sie stichpunktartig die Funktionsweise von Multi-Level Freedback-Queues. Berücksichtigen Sie dabei auch Anti-Aging. (5 Punkte)
